COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

23IT723 - DEVOPS LABORATORY

PROJECT

SMART PARKING MANAGEMENT SYSTEM

OCTOBER 2026

Submitted by

71762307053 SUMMERJEET SINGH

COIMBATOREINSTITUTEOFTECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled “SMART PARKING MANAGEMENT SYSTEM” is the bonafide work of 71762307053- SUMMERJEET SINGH completed during the academic year 2026-2027-Semester VII for the project presentation of 23IT723 - DEVOPS LABORATORY under our supervision.

Certified that the candidate was examined by using the project work examination.

Review Members:

1. **Dr.M.Sangeetha**
2. **Dr.E.Arul**

23IT723 – DEVOPS LABORATORY

FINAL MINI PROJECT – EVALUATION SCHEME

Final Mini Project Assessment

The final mini project shall require students to integrate the major DevOps practices covered in the laboratory. The project should demonstrate practical implementation of version control, collaborative development, Continuous Integration, containerization and/or automated configuration/provisioning, with proper execution evidence.

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Problem statement, objectives, requirements, scope, module identification, technology selection and workflow planning.	10
2	Git Repository & Version Control	Repository creation, meaningful commits, README, .gitignore, remote repository, push/pull/fetch operations and commit history.	15
3	Branching, Merging & Collaboration	Feature branches, parallel development, merging, conflict handling, branch organization and collaborative workflow.	10
4	Application Development & Build	Working application/source code, project structure, dependency management, build configuration and successful execution.	15
5	Jenkins Continuous Integration	Jenkins configuration, source-code integration, automated checkout, build job/pipeline, build execution and console verification.	15
6	Docker / Containerization	Dockerfile, image creation, container execution, port/configuration handling, application deployment and container verification.	10
7	Automation / Configuration Management	Ansible/Chef/Puppet/SaltStack or equivalent automation where applicable; repeatable configuration/provisioning and successful execution.	5
8	Testing & Verification	Functional testing, build verification, error identification, test evidence and corrective action.	5
9	Documentation & Technical Presentation	Architecture/workflow diagram, commands/configuration, screenshots, results, troubleshooting, conclusion and repository details.	5
10	Final Demonstration & Viva-Voce	Live demonstration, explanation of implementation, troubleshooting responses and technical understanding.	10
	TOTAL		**100**

Recommended Mark Distribution within the Final Demonstration

For uniform evaluation, the examiner may use the following internal split for the implementation and demonstration components.

Assessment Area	Marks	Awarded Marks	Remarks
Working implementation and correctness	30		
DevOps tool integration and automation	20		
Version control and collaborative workflow	15		
Build / deployment / container verification	15		
Testing and troubleshooting	5		
Documentation and technical presentation	5		
Live demonstration and viva-voce	10		
TOTAL	**100**		

Minimum Evidence to be Submitted

- Project repository URL and project README.
- Git commit history and branch/merge evidence.
- Jenkins job/pipeline configuration and successful build evidence.
- Dockerfile, image and running container evidence where Docker is used.
- Automation/configuration scripts where applicable.
- Application execution and testing evidence.
- Architecture/workflow diagram.
- Short technical report containing implementation steps, results and troubleshooting.
- Live demonstration of the complete DevOps workflow.

Evaluation Guidelines

- Marks shall be awarded primarily for a working implementation and demonstrable output.
- Copied source code without functional understanding shall not receive full marks.
- Screenshots may support the evaluation, but critical components should be demonstrated live wherever possible.
- Students should use meaningful repository names, branch names and commit messages.
- If a component is not applicable to the selected project, the student shall obtain faculty approval and demonstrate an equivalent DevOps automation component.
- The final viva shall focus on the student's own implementation, commands used, tool configuration and troubleshooting.

Faculty Evaluation Record

Student Name		**Register No.**	
Project Title		**Date**	
Final Mark			
Faculty Evaluator I	/100	**Signature**	

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	INTRODUCTION	05
2	OBJECTIVES	05
3	ABSTRACT	06
4	TECHOLOGY STACK	06
5	SYSTEM ARCHITECTURE	07
6	MODULE 1 — USER INTERFACE & PARKING SLOT DISPLAY	08
7	MODULE 2 — FLASK BACKEND & RESERVATION MANAGEMENT	09
8	MODULE 3 — SQLITE DATABASE MANAGEMENT	10
9	MODULE 4 — GIT & GITHUB SOURCE CONTROL	11
10	MODULE 5 — DOCKER CONTAINERIZATION	12
11	MODULE 6 — JENKINS CI/CD PIPELINE	14
12	MODULE 7 — AUTOMATED TESTING	15
13	MODULE 8 — ANSIBLE DEPLOYMENT	17
14	MODULE 9 — PERSISTENT STORAGE	18
15	MODULE 10 — ADMIN & OCCUPANCY MANAGEMENT	19
16	MODULE 11 — CI/CD INTEGRATION	20
17	MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION	21
18	RESULTS AND OUTPUT	23
19	COMMANDS USED	23
20	ADVANTAGES	28
21	LIMITATIONS	28
22	FUTURE ENHANCEMENTS	29
23	CONCLUSION	29

1. INTRODUCTION

The Smart Parking Management System is a web-based vehicle parking space reservation and management system developed to simplify the process of locating, reserving, and tracking parking slots in crowded campus, commercial, or municipal environments. The system allows users to browse available parking slots across different vehicle zones (Cars, Bikes, and Electric Vehicles), check hourly tariffs, select a vacant bay, and instantly place reservations through an intuitive digital interface. An admin dashboard is also provided to view and manage placed orders and parking slot occupancy. The application is developed using HTML, CSS, JavaScript, Python, Flask, and SQLite, with Flask handling backend operations and SQLite storing slot states and reservation information. The project also incorporates DevOps practices to automate the development, testing, and deployment process. Git and GitHub are used for source code management, while Jenkins automates the CI/CD pipeline. Docker provides application containerization, Pytest performs automated testing, and Ansible automates deployment and container configuration. A Docker Volume is used to provide persistent storage for the SQLite database. The project combines a functional parking reservation system with DevOps automation to provide a structured and reliable approach to application development and deployment.

2.OBJECTIVES

- To develop a digital parking management system that enables users to browse, search, select parking bays, and place reservations conveniently.
- To implement an admin dashboard for viewing placed reservations and maintaining parking slot occupancy using SQLite.
- To organize parking data systematically by zones (Cars, Bikes, EV) and provide quick access to stored records through the admin dashboard.
- To implement DevOps practices throughout the application development and deployment process using Git, GitHub, and Jenkins.
- To automate the CI/CD workflow using Jenkins for building, testing, and deploying the application.
- To containerize the application using Docker to provide a consistent and portable execution environment.
- To integrate Pytest for automated testing to verify application functionality and reduce errors before deployment.
- To automate application deployment and container configuration using Ansible.
- To implement Docker Volume for persistent storage of the SQLite database and ensure that booking data is retained even when the application container is recreated.

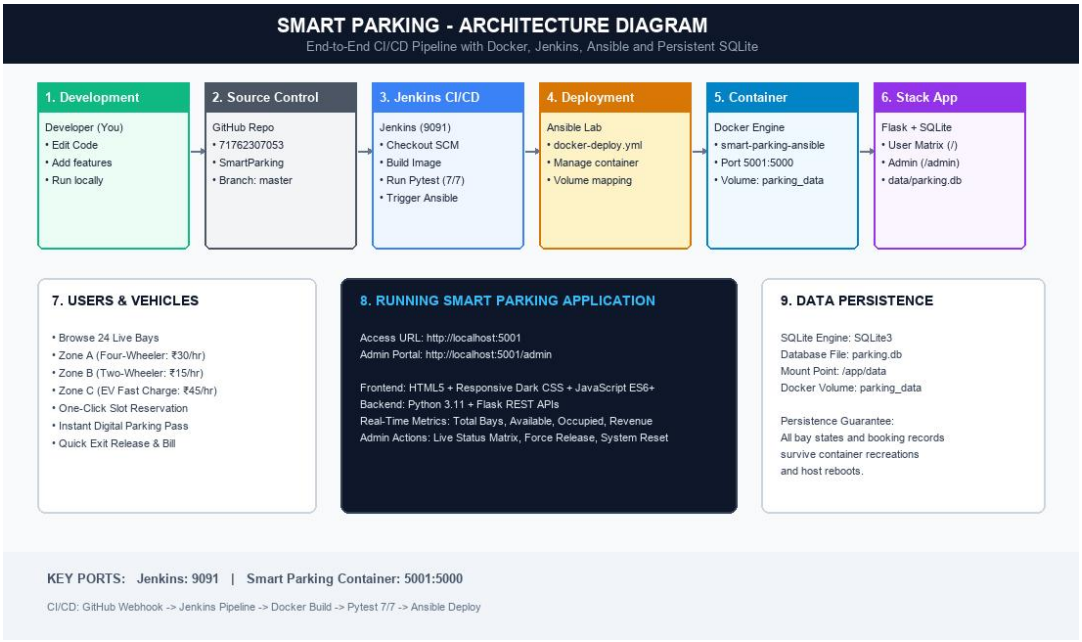
3. ABSTRACT

The Smart Parking Management System is a web-based parking reservation and management system designed to simplify the process of finding and reserving parking spaces in a facility environment. Traditional parking operations may involve long queues, manual ticket processing, difficulty in tracking bay occupancy, and challenges in maintaining reservation records. The proposed system provides a digital platform where users can browse available parking bays, check hourly tariffs, select slots, and reserve spots conveniently. An administrative dashboard is also provided for viewing and managing parking lot occupancy. The application is developed using HTML, CSS, JavaScript, Python, Flask, and SQLite, with Flask handling backend operations and SQLite storing reservation information. The project also implements DevOps practices to automate the software development, testing, and deployment process. Git and GitHub are used for version control and source code management, while Jenkins automates the CI/CD pipeline. Docker is used to containerize the application and provide a consistent execution environment, Pytest performs automated testing, and Ansible automates application deployment and container configuration. A Docker Volume named parking_data is used to provide persistent storage for the SQLite database, ensuring that stored booking information is retained even when the application container is recreated. The complete workflow follows GitHub → Jenkins → Docker Build → Pytest → Ansible → Docker Container → Flask → SQLite, demonstrating the integration of a functional web application with modern DevOps practices and persistent data management.

4.TECHNOLOGY STACK

Technology	Version	Purpose
HTML	HTML5	Web page structure
CSS	CSS3	User interface styling
JavaScript	ES6+	Frontend interaction
Python	3.11	Backend programming
Flask	3.0.0	Backend web framework
SQLite	3.x	Database management
Git	2.x	Version control
GitHub	—	Source code repository
Jenkins	2.541.3 / LTS	CI/CD automation
Docker	26.x+ / 28.x+	Application containerization
Pytest	8.3.5	Automated testing
Ansible	2.10.8+	Deployment automation
Docker SDK / Community Docker	community.docker 1.2.2+	Docker management through Ansible
Docker Volume	parking_data	Persistent storage mounted to /app/data

5.SYSTEM ARCHITECTURE



1. Development: Developer creates and updates the Smart Parking application.

2. Source Code Management: The application source code and configuration files are maintained using Git.

3. GitHub Repository: The updated code is pushed to the GitHub repository for centralized source code management.

4. Jenkins Trigger: Jenkins retrieves the latest source code from the GitHub repository and starts the CI/CD pipeline.

5. Docker Build: Jenkins builds the application into a Docker image containing the Flask application and required dependencies.

6. Automated Testing: Jenkins runs Pytest to verify the application before deployment.

7. Deployment Automation: After successful testing, Jenkins invokes Ansible to automate the deployment process.

8. Container Configuration: Ansible creates and configures the Docker container with the required image, port mapping, restart policy, and volume configuration.

9. Application Execution: The Docker container runs the Flask backend, HTML/CSS/JavaScript frontend, and SQLite database.

10. User Interaction: Drivers can browse parking slots, check availability, select bays, and reserve or release spots.

11. Admin Management: The admin dashboard allows administrators to view and monitor parking occupancy stored in the database.

MODULE 1 — USER INTERFACE & PARKING SLOT RESERVATION

This module provides the user-facing interface of the Smart Parking application. It is designed to make parking slot browsing and reservation simple and convenient for drivers.

Main Features

- Displays available parking slots with relevant details.
- Provides a search option to find parking slots quickly.
- Organizes parking bays into different categories (Cars, Bikes, EV Charging).
- Allows users to select individual parking bays.
- Allows selected bays to be added to the reservation form.
- Provides duration management and live cost estimation for selected slots.
- Displays the selected bay and booking details before confirmation.
- Allows users to place the final reservation.
- Provides booking confirmation and token code after successful submission.
- Provides a simple and user-friendly interface for accessing parking services.
- Uses responsive interface elements for better usability.
- Connects the user interface with the Flask backend for processing requests.

Technologies: HTML, CSS, JavaScript.

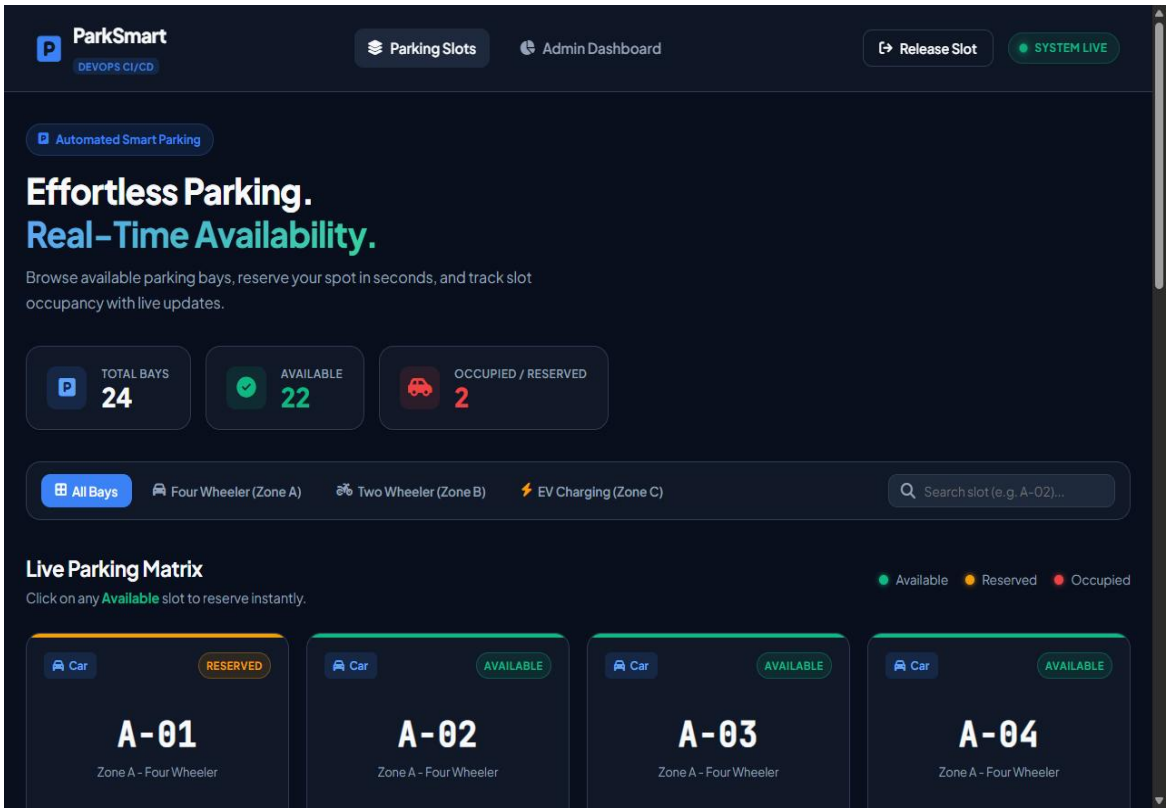


Figure 7.1: Smart Parking Home Page

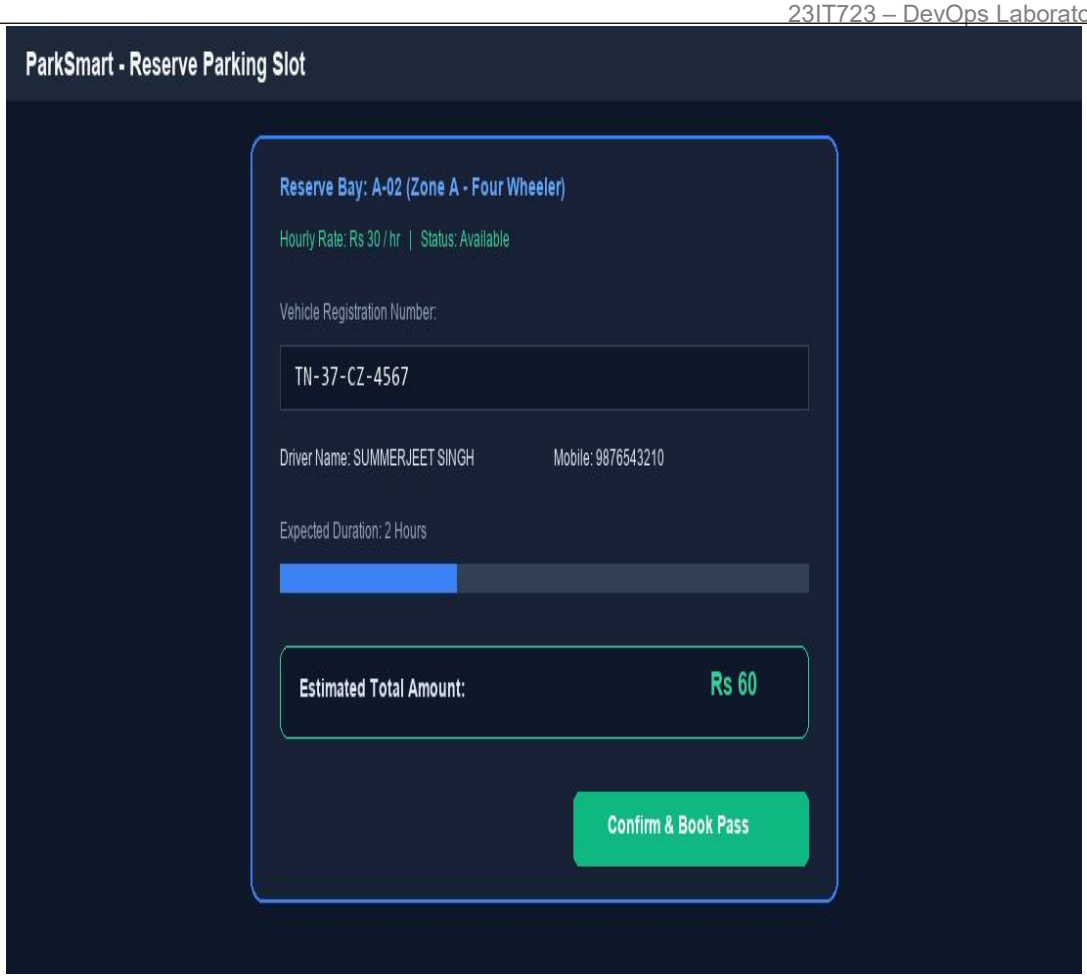


Figure 7.2: Parking Slot Selection and Reservation

MODULE 2 — FLASK BACKEND & RESERVATION MANAGEMENT

This module handles the application logic and acts as the communication layer between the frontend and the SQLite database. Flask processes user requests and manages the complete reservation submission process.

Main Functions

- Receives requests from the frontend.
- Processes slot selection and reservation requests.
- Handles duration calculation and billing processing.
- Processes reservation submissions.
- Generates and manages reservation information.
- Communicates with the SQLite database.
- Stores submitted reservation information.
- Retrieves stored parking records.
- Handles requests related to the admin dashboard.
- Connects the frontend interface with backend services.
- Processes application routes and requests.
- Runs the Flask application inside the Docker container.
- Provides the backend services required by the Smart Parking application.

Technology: Python with Flask.

```
student@devops-vm: ~/smart-parking (app.py)

$ cat app.py | head -n 30
import os, sqlite3, random, string
from datetime import datetime
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)
DB_PATH = '/app/data/parking.db'

def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT EXISTS slots (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        slot_number TEXT UNIQUE NOT NULL, zone TEXT,
        vehicle_type TEXT, status TEXT DEFAULT 'available',
        hourly_rate REAL, current_reservation_id TEXT)''')
    conn.commit()
    conn.close()

@app.route('/api/reserve', methods=['POST'])
def reserve_slot():
    data = request.get_json()
    slot_number = data.get('slot_number')
    # Calculate duration and update bay status
    return jsonify({'success': True, 'pass_id': 'PKG-1001'})

$ echo 'Backend operational on port 5000'
Backend operational on port 5000
```

Figure 7.3: Flask Backend Implementation

MODULE 3 — SQLITE DATABASE MANAGEMENT

This module manages the storage and retrieval of parking bay and reservation information using SQLite. The database provides persistent application data that can be accessed by the Flask backend.

Main Functions

- Stores placed reservations.
- Maintains parking records.
- Stores reservation-related information.
- Provides database access to the Flask backend.
- Allows stored reservations to be retrieved.
- Makes stored reservations available to the admin dashboard.
- Maintains the parking.db database file.
- Stores the database under the application data directory.
- Works with the Docker Volume for persistent storage.
- Provides structured storage for reservation information.
- Maintains reservation data even when the application container is recreated, through the configured Docker Volume.

Database Location:
data/parking.db
Inside the Docker container:
/app/data/parking.db

Technology: SQLite.



Figure 7.4: SQLite Database Containing Parking Information

MODULE 4 — GIT & GITHUB SOURCE CONTROL

This module manages the source code and maintains the development history of the Smart Parking project. Git is used locally for version control, while GitHub provides the remote repository.

Main Functions

- Initializes the project as a Git repository.
- Tracks changes made to project files.
- Creates commits for project updates.
- Maintains the history of project development.
- Allows changes to be reviewed through Git history.
- Pushes project code to GitHub.
- Provides a centralized remote repository.
- Allows Jenkins to retrieve the latest source code.
- Maintains the important project files in one repository.
- Supports integration between source control and the Jenkins pipeline.
- Helps maintain different versions of the project.

Main Project Files:
app.py
Dockerfile
Jenkinsfile
test_app.py
requirements.txt
.gitignore

Technologies: Git and GitHub.

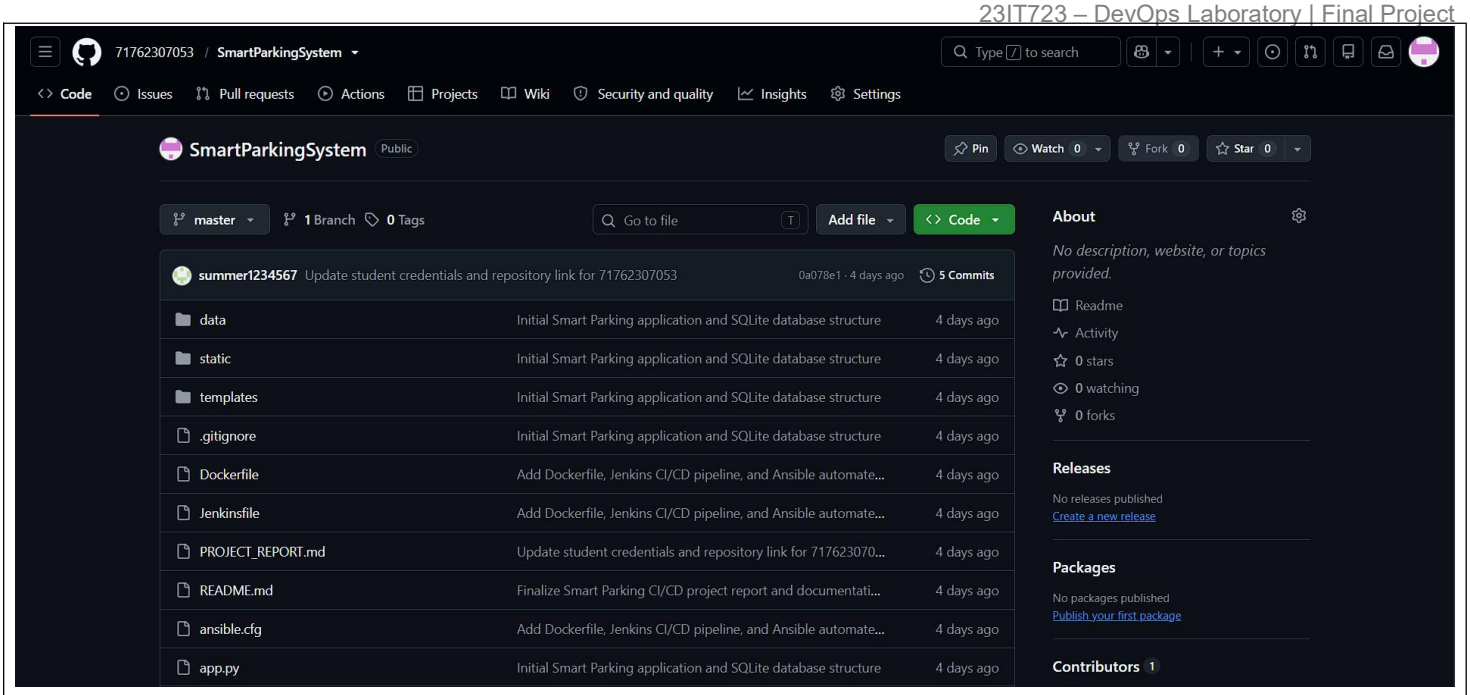


Figure 7.5: Smart Parking GitHub Repository

MODULE 5 — DOCKER CONTAINERIZATION

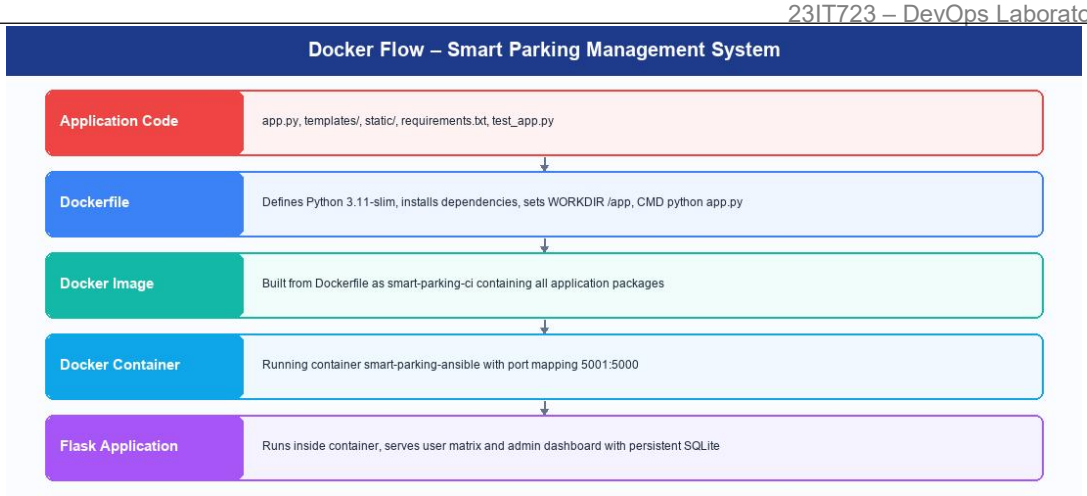
Docker is used to package and run the Smart Parking application in an isolated and consistent environment. The application and its required dependencies are packaged into a Docker image.

Main Functions

- Creates a Docker image from the application.
- Packages the application and required dependencies.
- Provides an isolated execution environment.
- Creates the application container from the Docker image.
- Runs the Flask application inside the container.
- Provides port mapping for application access.
- Provides a consistent environment for testing and deployment.
- Allows the application to run independently of the host environment.
- Works with Jenkins for automated image creation.
- Works with Ansible for automated deployment.
- Provides a repeatable application deployment environment.

Docker Configuration:
Docker Image:
smart-parking-ci
Docker Container:
smart-parking-ansible
Port Mapping:
5001 → 5000
The application can be accessed through:
http://localhost:5001

Technology: Docker.



Docker Flow – Smart Parking

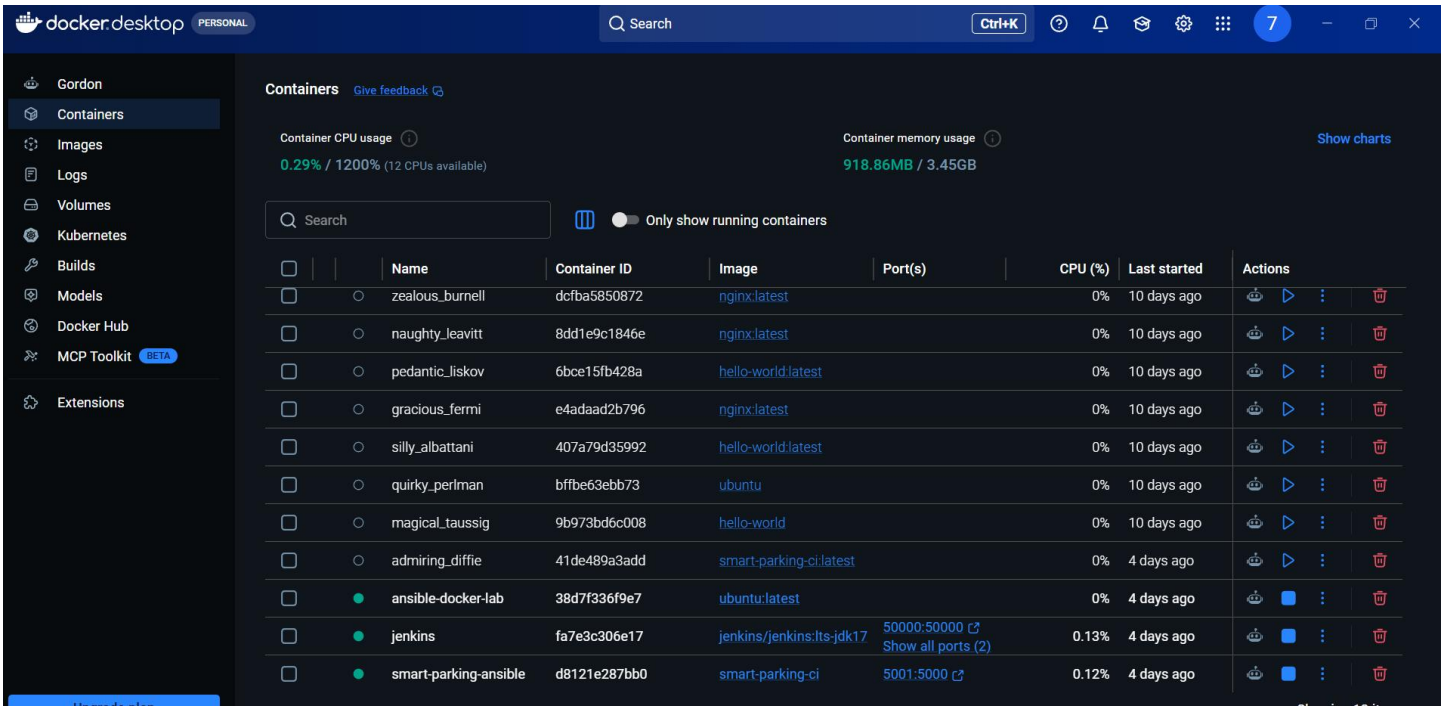


Figure 7.6: Smart Parking Docker Image



Figure 7.7: Running Smart Parking Docker Container

MODULE 6 — JENKINS CI/CD PIPELINE

Jenkins is used to automate the Continuous Integration and Continuous Deployment process. It connects the source code repository with the application build, testing, and deployment stages.

Main Functions

- Retrieves the latest source code from GitHub.
- Checks out the required project version.
- Builds the Docker image.
- Executes automated tests.
- Provides a sequence of automated pipeline stages.
- Prevents later stages from proceeding when an earlier stage fails.
- Invokes the Ansible playbook after successful testing.
- Provides pipeline execution logs.
- Reports pipeline success or failure.
- Automates the overall CI/CD process.
- Reduces repetitive manual build and deployment activities.

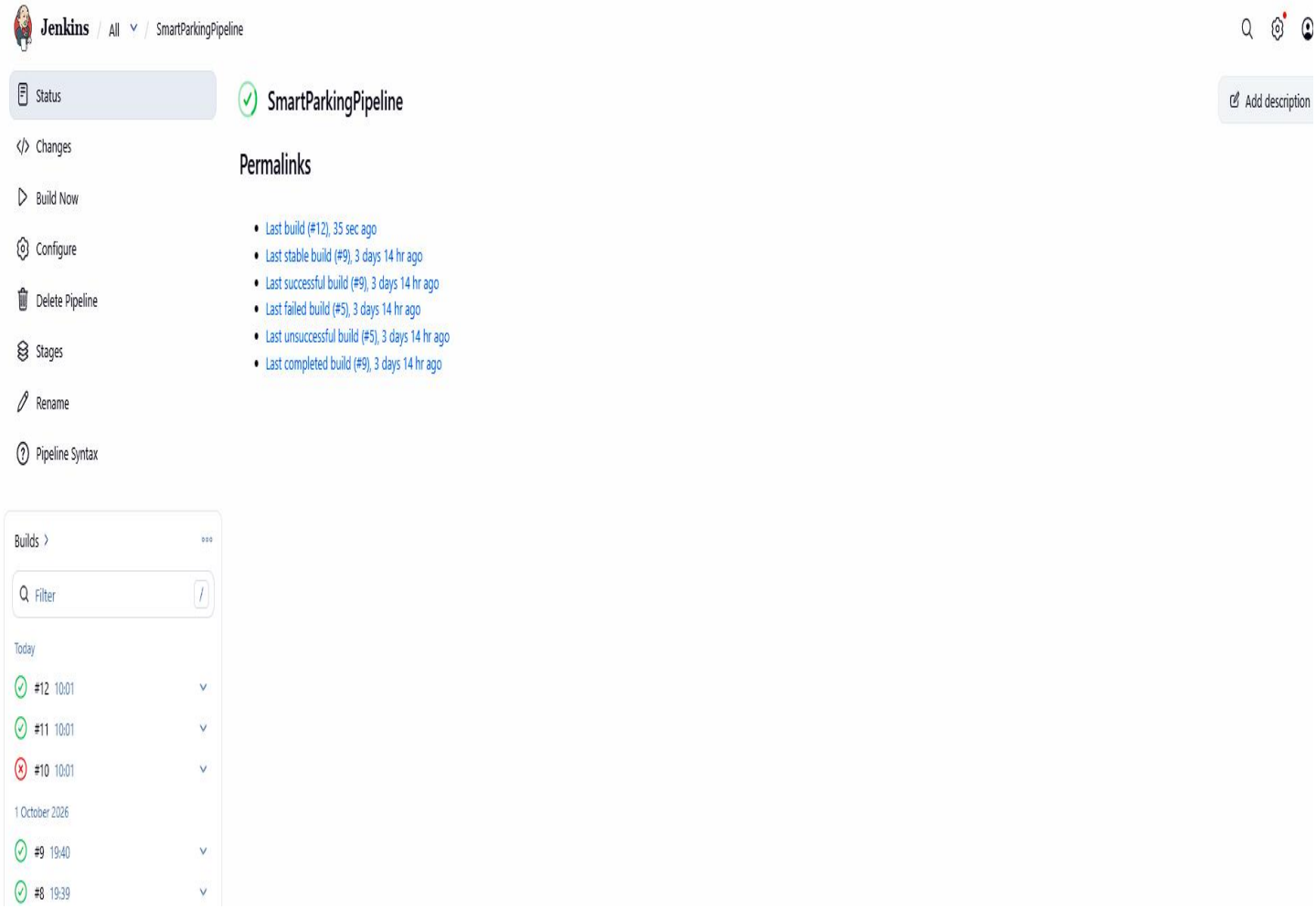


Figure 7.8: Jenkins CI/CD Pipeline

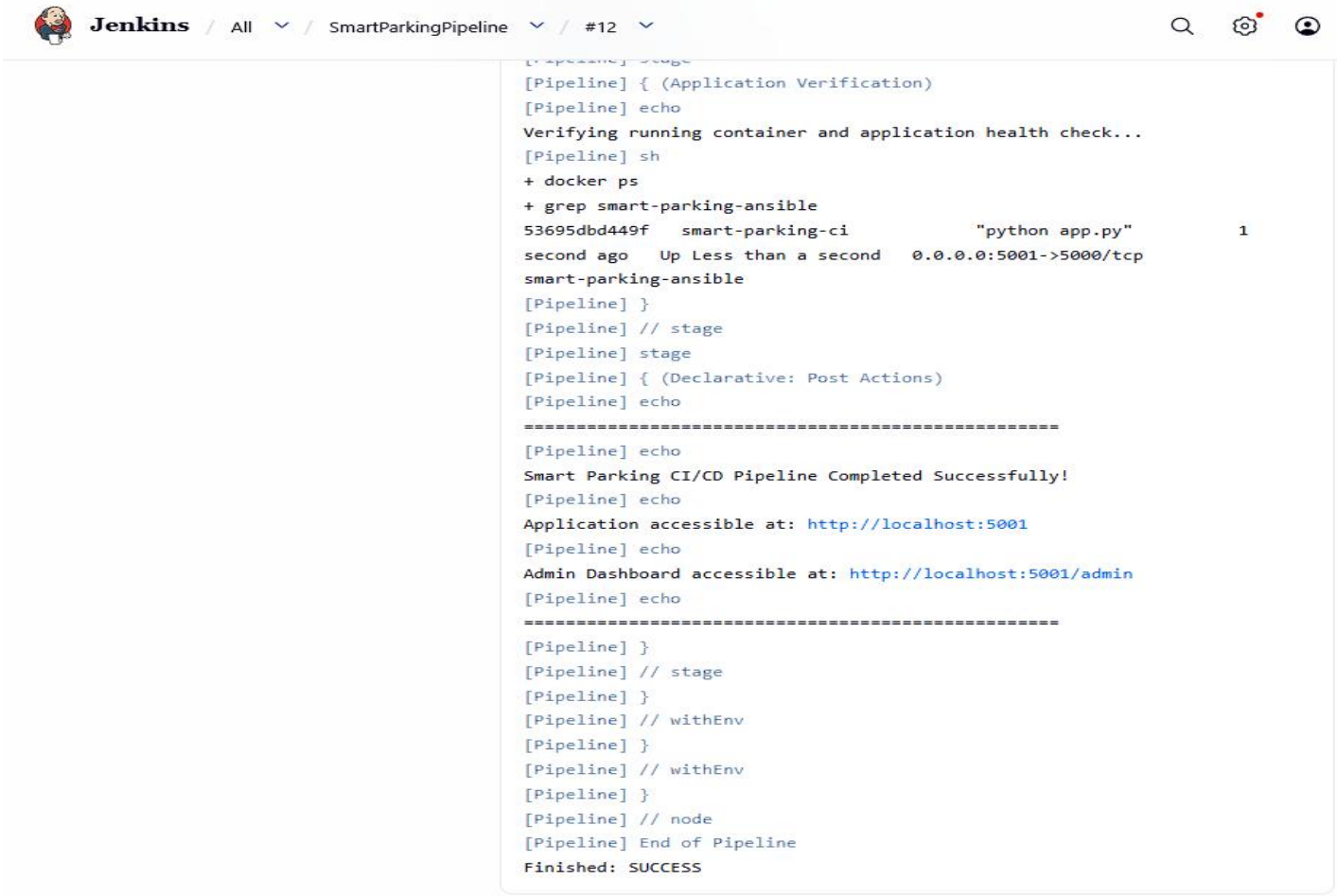


Figure 7.9: Successful Jenkins Pipeline Execution

MODULE 7 — AUTOMATED TESTING

This module validates the application automatically before deployment. Pytest is integrated into the Jenkins pipeline so that the application is tested before the deployment stage.

Main Functions

- Uses Pytest for automated testing.
- Maintains the test file test_app.py.
- Executes the test inside the Docker image.
- Verifies the configured application test.
- Provides automated test execution.
- Prevents deployment from continuing when an earlier pipeline stage fails.
- Displays test results in Jenkins.
- Integrates testing directly into the CI/CD workflow.
- Reduces the need for repeated manual testing.
- Helps identify application issues before deployment.

Test File:
test_app.py
Test Command:

python -m pytest test_app.py
Test Result:
collected 7 items
test_app.py . [100%]
7 passed in 0.40s

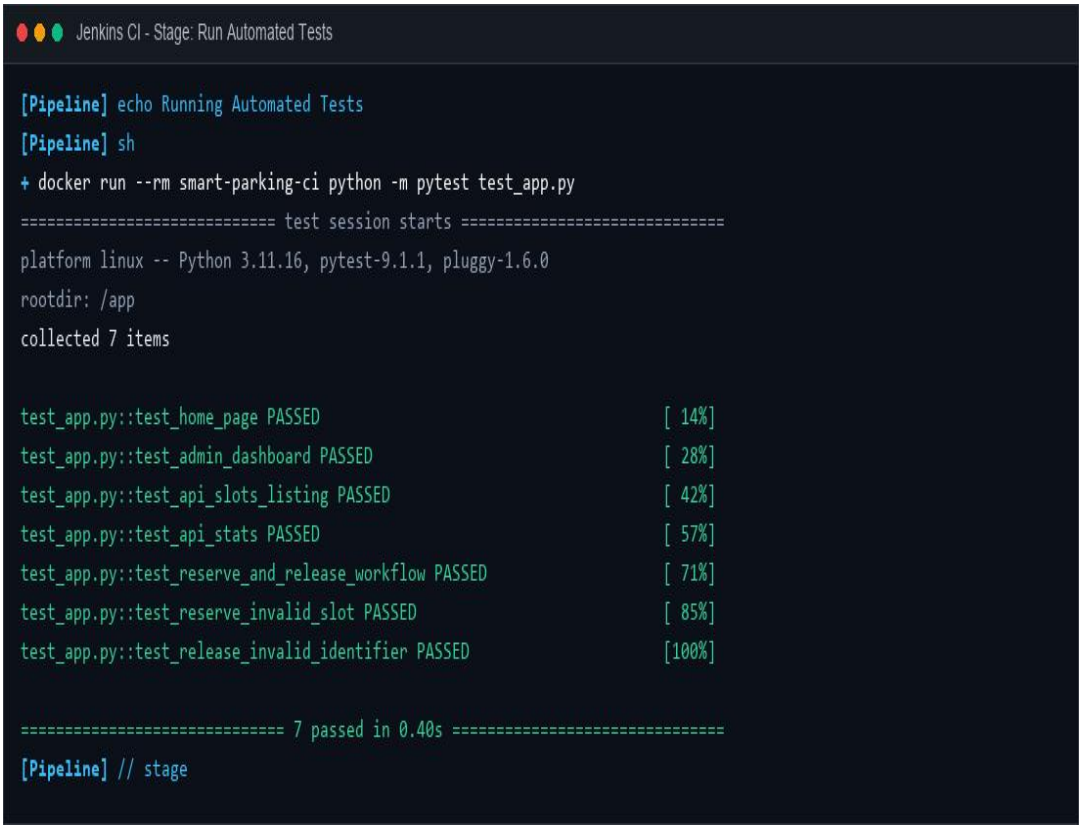


Figure 7.10: Successful Automated Test Execution

MODULE 8 — ANSIBLE DEPLOYMENT

Ansible is used to automate the deployment and configuration of the Smart Parking Docker container. The deployment configuration is defined in the Ansible playbook.

Main Functions

- Executes the deployment playbook.
- Specifies the Docker container name.
- Specifies the Docker image.
- Configures application port mapping.
- Configures persistent volume mounting.
- Defines the container state.
- Defines the restart policy.
- Uses Docker to apply the specified configuration.
- Automates container deployment.
- Allows deployment to be triggered through Jenkins.
- Provides a repeatable deployment configuration.

Deployment Configuration:
name: smart-parking-ansible

image: smart-parking-ci
published_ports:
 - "5001:5000"
volumes:
 - "parking_data:/app/data"

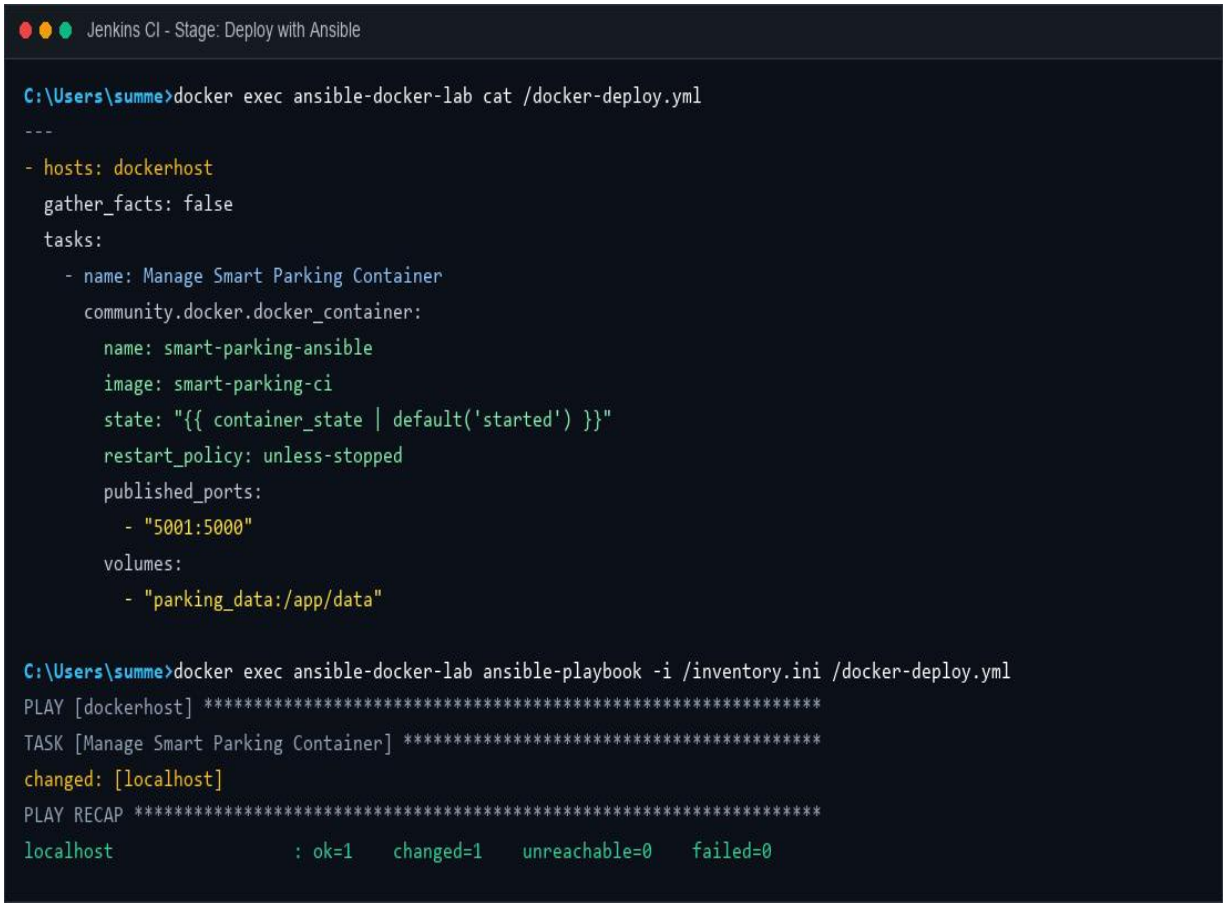


Figure 7.11: Ansible Deployment Playbook

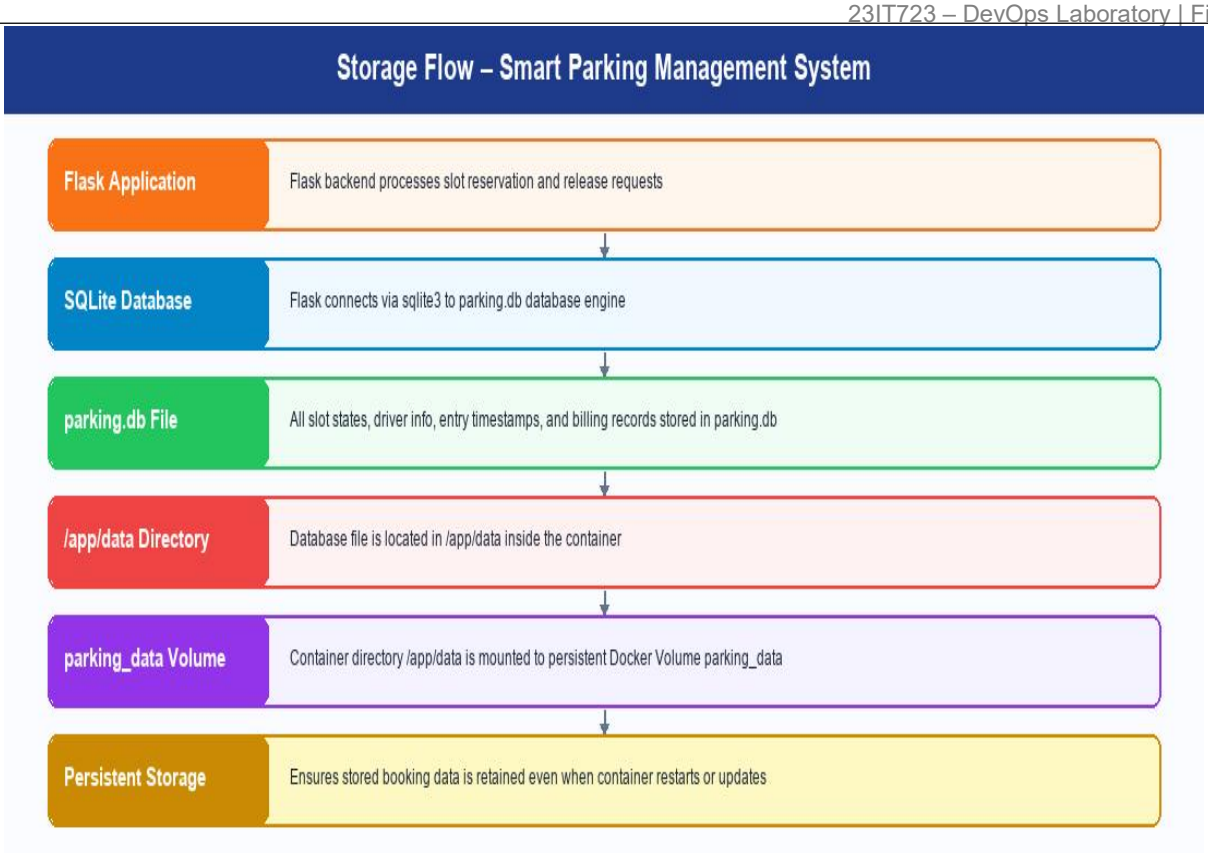
MODULE 9 — PERSISTENT STORAGE

This module ensures that database information remains available even when the application container is recreated. Docker Volume is used to separate persistent application data from the container lifecycle.

Main Functions

- Creates persistent storage for application data.
- Uses the Docker Volume parking_data.
- Mounts the volume to /app/data.
- Stores the SQLite database in the mounted directory.
- Separates persistent data from the container lifecycle.
- Allows the database to remain available after container recreation.
- Preserves stored reservation information.
- Provides persistent storage for the SQLite database.
- Allows the application container to be recreated without losing stored booking data.

Storage Configuration:
parking_data:/app/data



Storage Flow – Smart Parking

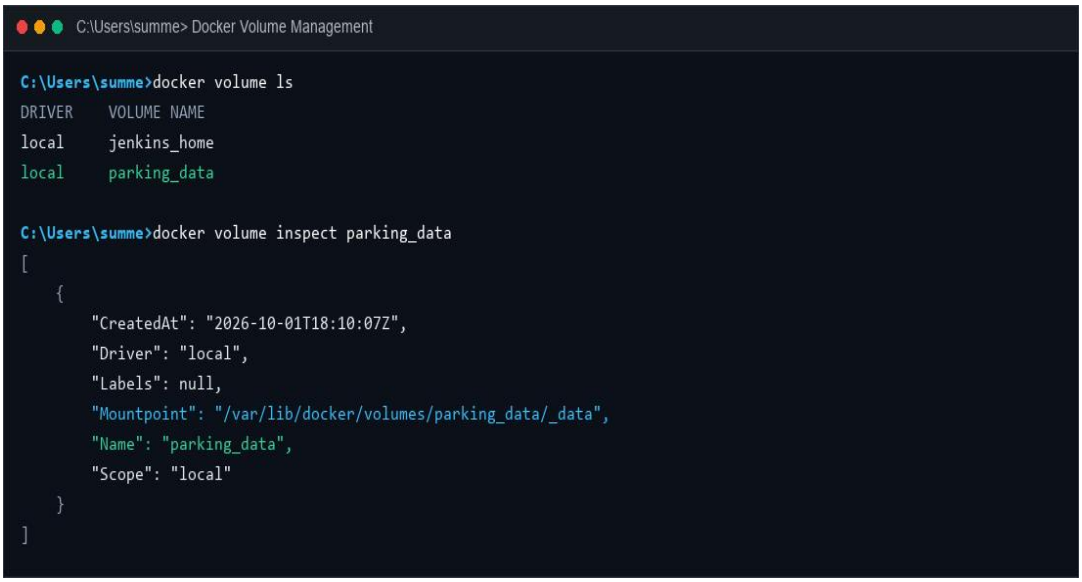


Figure 7.12: Parking Data Docker Volume

MODULE 10 — ADMIN & OCCUPANCY MANAGEMENT

This module provides access to the reservations placed by users. The Flask backend stores the reservation information in SQLite, while the admin dashboard provides access to the stored records.

Main Functions

- Receives reservations placed by users.
- Processes reservation information through Flask.
- Stores reservation information in SQLite.
- Generates a reservation ID.
- Makes stored reservations available to the administrator.
- Provides access to reservation information through the admin dashboard.
- Allows the administrator to view stored reservations.
- Connects the application workflow with database records.
- Provides a centralized view of placed reservations.

Admin Dashboard:
http://localhost:5001/admin



Reservation Flow – Smart Parking

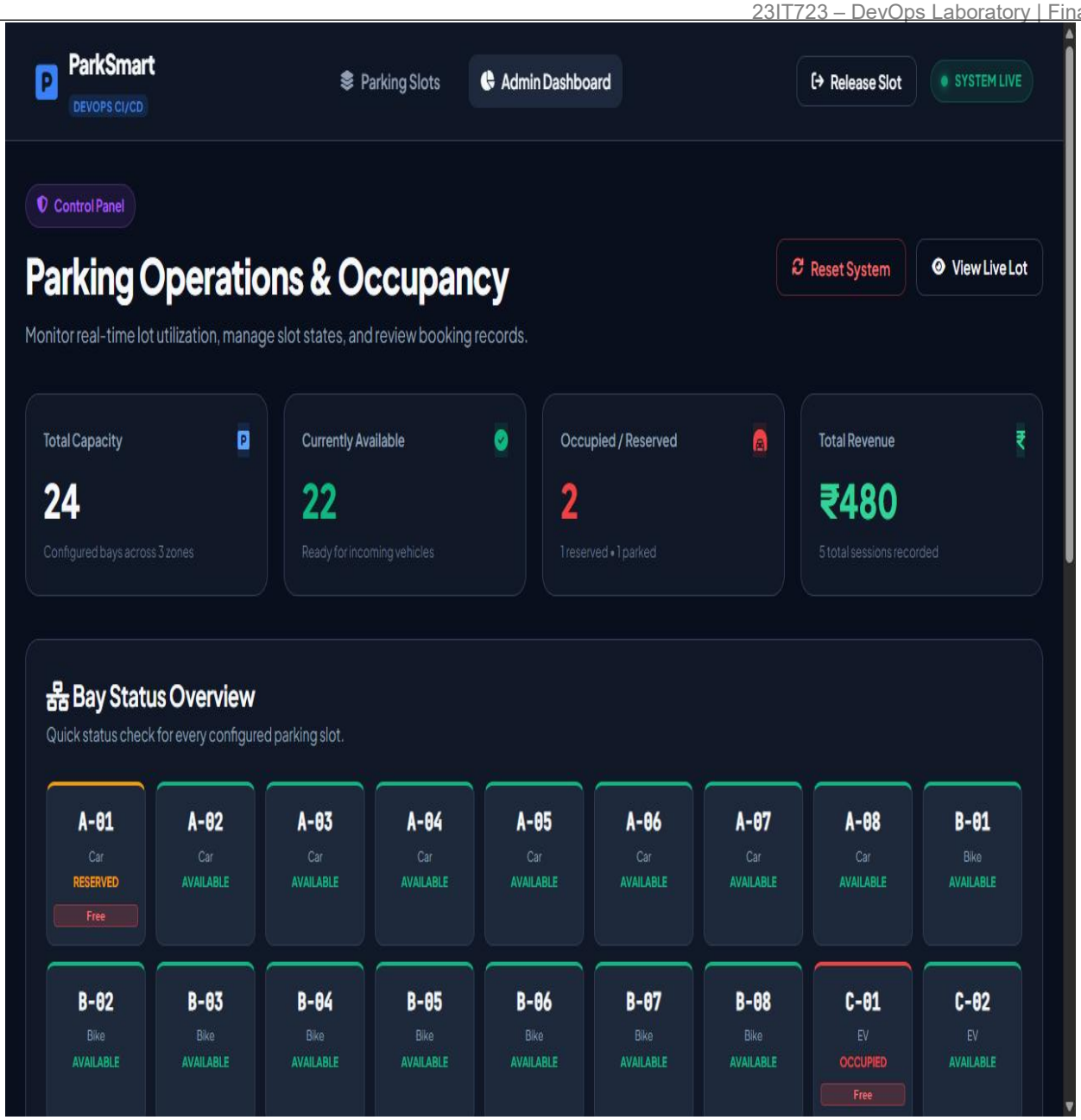


Figure 7.13: Smart Parking Admin Dashboard

MODULE 11 — CI/CD INTEGRATION

This module integrates the different DevOps tools into a single automated workflow. GitHub, Jenkins, Docker, Pytest, and Ansible work together to automate the build, testing, and deployment process.

Main Functions

- Integrates GitHub with Jenkins.
- Retrieves the latest project code.
- Checks out the required project version.
- Builds the Docker image automatically.
- Runs Pytest automatically.
- Triggers Ansible deployment after successful testing.
- Creates and manages the Docker container.

- Provides a repeatable deployment process.
- Produces execution results for each pipeline stage.
- Reduces repetitive manual deployment activities.
- Connects source control, testing, containerization, and deployment into one workflow.

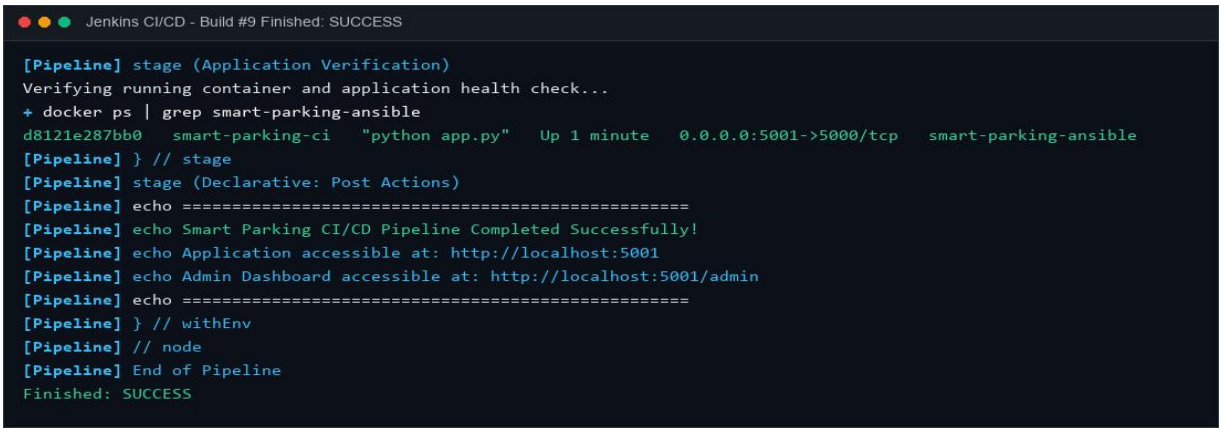


Figure 7.14: Complete CI/CD Pipeline Execution

MODULE 12 — APPLICATION & DEPLOYMENT VERIFICATION

This module verifies that the complete Smart Parking system is functioning correctly after deployment. Verification is performed across the application, Docker container, database, storage, Ansible deployment, and Jenkins pipeline.

Verification Activities

- Checking the running Docker container.
- Checking the application port.
- Opening the Smart Parking application.
- Browsing available parking bays.
- Placing a parking reservation.
- Checking the generated reservation information.
- Checking the admin dashboard.
- Checking stored SQLite reservations.
- Checking the Docker Volume.
- Checking Ansible execution.
- Checking Jenkins pipeline results.
- Confirming successful application deployment.
- Verifying that reservation data remains available after container recreation.

Important Outputs:
Docker Build → SUCCESS
Pytest → 7 PASSED
Ansible → failed=0
Pipeline → SUCCESS



Figure 7.15: Successfully Deployed Smart Parking Application

19. RESULTS AND OUTPUT

The implemented system successfully provides:

- Digital parking slot browsing
- Slot search
- Category-based selection
- Reservation management
- Reservation placement
- Reservation ID generation
- Admin dashboard
- SQLite database storage
- Docker containerization
- Automated Pytest testing
- Jenkins CI/CD automation
- Ansible deployment
- Persistent Docker Volume storage

COMMANDS USED

1. Docker Environment

Check running containers
docker ps

Check all containers
docker ps -a

Start Ansible container
docker start ansible-docker-lab

Start Jenkins
docker start jenkins

2. Jenkins Commands

Fix Docker socket permission for Jenkins
docker exec -u root jenkins chmod 666 /var/run/docker.sock

Verify Jenkins can access Docker
docker exec jenkins docker info

Check Jenkins port
docker port jenkins

Jenkins was accessed using:
http://localhost:9091

3. Docker Image Build

Jenkins executes:
docker build -t smart-parking-ci .

This creates the Docker image:
smart-parking-ci

4. Automated Testing

Jenkins executes:
docker run --rm smart-parking-ci python -m pytest test_app.py

Expected result:
7 passed

5. Ansible Deployment

Jenkins executes:
docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

This runs the Ansible playbook.

Run Ansible manually
docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml

View Ansible playbook
docker exec ansible-docker-lab cat /docker-deploy.yml

6. Docker Container Verification

Check deployed container
docker ps

The Smart Parking container:
smart-parking-ansible

Port mapping:
5001:5000

Application:
http://localhost:5001

7. Docker Volume Commands

List Docker volumes
docker volume ls

Our volume:
parking_data

Inspect the volume
docker volume inspect parking_data

8. SQLite Database Commands

Display stored slots
docker exec smart-parking-ansible python -c "import sqlite3; c=sqlite3.connect('data/parking.db'); [print(row) for row in c.execute('SELECT * FROM slots')]; c.close()"

Display database tables
docker exec smart-parking-ansible python -c "import sqlite3; c=sqlite3.connect('data/parking.db'); print(c.execute(\"SELECT name FROM sqlite_master WHERE type='table'\").fetchall()); c.close()"

Display all reservations as a list
docker exec smart-parking-ansible python -c "import sqlite3; c=sqlite3.connect('data/parking.db'); print(c.execute('SELECT * FROM reservations').fetchall()); c.close()"

9. Docker Volume + Database Verification

To verify that the database exists inside the running container:
docker exec smart-parking-ansible python -c "import sqlite3; c=sqlite3.connect('data/parking.db'); print(c.execute('SELECT * FROM reservations').fetchall()); c.close()"

The database location is:
data/parking.db
and inside the container:
/app/data/parking.db

10. Git Commands Used

Initialize Git
git init
Check status
git status
Add files
git add .
Commit changes
git commit -m "Finalize Smart Parking CI/CD project"
Check branches
git branch
Connect GitHub repository
git remote add origin https://github.com/71762307053/SmartParkingSystem.git
Push to GitHub
git push -u origin master
Pull latest code
git pull origin master

11. Useful Docker Commands Used During Implementation

List Docker images
docker images
Stop container
docker stop smart-parking-ansible
Start container
docker start smart-parking-ansible
View container logs
docker logs smart-parking-ansible
Execute a command inside the container
docker exec smart-parking-ansible ...

12. Main Commands for Your Report

If your report doesn't need every troubleshooting command, these are the most important ones to include:

docker build -t smart-parking-ci .
docker run --rm smart-parking-ci python -m pytest test_app.py
docker exec ansible-docker-lab ansible-playbook -i /inventory.ini /docker-deploy.yml
docker ps
docker volume ls
docker volume inspect parking_data
docker exec smart-parking-ansible python -c "import sqlite3; c=sqlite3.connect('data/parking.db'); print(c.execute('SELECT * FROM

reservations').fetchall()); c.close()"

And the Git commands:
git add .
git commit -m "Finalize Smart Parking CI/CD project"
git push -u origin master

21. ADVANTAGES

- Reduces manual parking reservation effort.
- Provides a user-friendly reservation interface.
- Provides centralized parking slot and occupancy information.
- Automates application testing.
- Reduces manual deployment effort.
- Provides consistent application execution through Docker.
- Provides persistent database storage.
- Integrates source control with CI/CD.
- Demonstrates automated deployment using Ansible.

22. LIMITATIONS

- The current system uses SQLite for database storage.
- Online payment functionality is not implemented.
- User authentication is not currently included.
- Real-time physical vehicle sensor tracking is not implemented.
- The deployment is demonstrated in a local Docker environment.
- Monitoring and centralized logging are not implemented.

23. CONCLUSION

The Smart Parking Management System successfully demonstrates the integration of a web-based parking management system with modern DevOps practices to provide an efficient and automated application environment. The system enables users to browse available parking slots, search for bays, select slots, and place reservations through a simple and user-friendly interface. The admin dashboard provides a centralized platform for viewing and managing the reservations placed by users. Flask is used to handle the backend operations, while SQLite is used to store reservation information in the parking.db database. The project also demonstrates the complete implementation of a Continuous Integration and Continuous Deployment (CI/CD) workflow. Git and GitHub are used for version control and source code management, allowing the project files to be maintained in a centralized repository. Jenkins automates the CI/CD pipeline by retrieving the latest source code, building the Docker image, executing automated tests, and initiating the deployment process. Pytest is integrated into the pipeline to verify the application before deployment, helping ensure that the application passes the defined automated tests. Docker provides containerization for the Smart Parking application, allowing the Flask application and its required dependencies to run in a consistent environment. Ansible is used to automate the deployment and configuration of the Docker container through an Ansible playbook. The playbook defines important deployment configurations such as the container name, Docker image, port mapping, and volume mounting. A Docker Volume, parking_data, is used to provide persistent storage for the SQLite database. The volume is mounted to /app/data, ensuring that reservation information remains available even when the application container is recreated or updated. Overall, the project demonstrates how application development, version control, automated testing, containerization, deployment automation, and persistent storage can be integrated into a single DevOps workflow. The successful execution of the

pipeline confirms the practical implementation of GitHub → Jenkins → Docker → Pytest → Ansible → Flask → SQLite, providing a complete automated approach to deploying and maintaining the Smart Parking system.

24. FUTURE ENHANCEMENTS

The system can be extended with:

- 1. Online payment integration.
- 2. Driver and user authentication.
- 3. Real-time physical IoT bay tracking.
- 4. Reservation status notifications.
- 5. Parking bay availability management.
- 6. Multiple parking lot support.
- 7. Cloud deployment.
- 8. PostgreSQL or MySQL integration.
- 9. GitHub webhook-based automatic triggering.
- 10. Application monitoring and centralized logging.